

Guia de Documentação do Projeto

Simulador do Modelo OSI (Python)

Comunicação de Dados

Apresentação

Este guia orienta o conteúdo esperado em cada documento da entrega do projeto. A entrega é composta por três tipos de documento, cada um com um propósito específico:

Documento	Pergunta que responde	Local
README	"O que é isso e por onde começo?"	Raiz do repositório
Tutoriais	"Como eu executo e uso isso?"	<code>docs/</code>
Documentação	"Como isso funciona internamente?"	<code>docs/</code>

A documentação é parte da entrega, não um acessório dela. O projeto é avaliado por quem **não** participou do desenvolvimento e abrirá o programa com dois cliques, sem montar ambiente nem digitar comando algum. Tudo o que for necessário para usar e entender o simulador precisa estar escrito.

1 README

O README é o ponto de entrada do repositório. É o primeiro arquivo lido por quem chega ao projeto e deve dar uma visão geral rápida do que está ali e de como navegar. Deve conter, no mínimo:

- Título do projeto;
- Disciplina, semestre e nome do(a) professor(a);
- Autoria;
- Breve descrição do projeto (1 a 2 parágrafos);
- **Como executar:** qual arquivo abrir com dois cliques. Esta informação vem antes de qualquer outra, porque é a primeira coisa que quem avalia vai procurar;
- **Estrutura do repositório:** árvore de pastas explicando o que há em cada lugar;
- Lista dos arquivos de código com explicação breve do que cada um faz;
- Requisitos de ambiente: versão do Python e bibliotecas utilizadas, se houver;
- **Funcionalidades:** lista do que o simulador faz, com indicação do arquivo em que cada funcionalidade está implementada;
- Atalhos para os tutoriais e para a documentação dentro de `docs/`;
- **Por onde começar:** trilha de leitura indicando a ordem dos documentos.

Exemplo de estrutura do README

```
# Simulador do Modelo OSI

Projeto pratico da disciplina de Comunicacao de Dados,
ministrada por <nome do professor>. Semestre <ano/semestre>.

## Autoria
- Nome

## Descricao
Simulador do modelo OSI em uma rede com tres redes locais,
quatro roteadores e cinco computadores. Exibe passo a passo o
encapsulamento, a troca de enderecos fisicos a cada salto e a
decisao de rota em cada roteador.

## Como executar
Clique duas vezes em 'SimuladorOSI.exe'.
Nao e necessario instalar nada.

## Estrutura do repositorio
simulador_osi/
|-- SimuladorOSI.exe   Programa pronto para executar
|-- topologia.json     Rede simulada, editavel
|-- main.py            Ponto de entrada do codigo-fonte
|-- simulador/        Codigo-fonte do simulador
'-- docs/              Tutoriais e documentacao tecnica

## Arquivos de codigo
- 'simulador/pdu.py'      - unidade de dados e cabecalhos
- 'simulador/camadas.py' - as sete classes de camada
- 'simulador/dispositivos.py' - computador e roteador
- 'simulador/rede.py'    - enlaces, rotas e topologia
- 'simulador/motor.py'   - laco de simulacao e interface

## Funcionalidades
| O que faz | Onde |
|-----|-----|
| Encapsulamento e desencapsulamento | 'simulador/camadas.py' |
| Encaminhamento por menor custo      | 'simulador/rede.py' |
| Segmentacao e remontagem             | 'simulador/camadas.py' |
| Registro de eventos e eficiencia     | 'simulador/motor.py' |

## Documentacao
- [Tutorial de execucao](./docs/tutorial_execucao.pdf)
- [Tutorial de uso](./docs/tutorial_uso.pdf)
- [Documentacao tecnica](./docs/documentacao_projeto.pdf)

## Por onde comecar
1. Abra o programa e siga o tutorial de execucao
2. Reproduza o cenario E2 pelo tutorial de uso
3. Consulte a documentacao tecnica para entender o codigo
```

2 Tutoriais (docs/tutorial_*.pdf)

Os tutoriais são documentos **operacionais**: explicam **como fazer**, passo a passo. Não precisam justificar decisões nem explicar conceitos, apenas guiar a execução. Devem ser autoexplicativos a ponto de outra pessoa conseguir usar o simulador sem precisar perguntar nada.

2.1 tutorial_execucao.pdf

Como colocar o programa para funcionar. É o documento mais curto e o mais importante: se ele falhar, nada mais será avaliado.

Roteiro mínimo do tutorial de execução

1. **Pré-requisitos:** sistema operacional e, caso a entrega não seja um executável independente, versão do Python e bibliotecas necessárias;
2. **Abertura:** qual arquivo receber o duplo clique, com captura de tela da pasta descompactada;
3. **Primeira tela:** captura da janela inicial, identificando cada campo e cada botão;
4. **Execução mínima:** a sequência mais curta possível que produz um resultado na tela, para confirmar que o programa funciona;
5. **Resultado esperado:** captura do que deve aparecer ao final dessa sequência;
6. **Problemas conhecidos:** o que fazer se a janela não abrir ou fechar sozinha.

2.2 tutorial_uso.pdf

Como operar o simulador em cada uma de suas funções. Este tutorial descreve o programa do ponto de vista de quem o usa, e não de quem o escreveu.

Roteiro mínimo do tutorial de uso

1. **Escolher o cenário:** onde se seleciona entre os sete cenários de validação do enunciado;
2. **Definir origem, destino e mensagem:** como escolher o computador de origem, o de destino, os processos envolvidos e o texto a ser enviado;
3. **Trocar a rede simulada:** como substituir o arquivo de topologia por outro, sem gerar um novo executável;
4. **Executar a simulação:** onde ficam os controles de passo a passo, execução contínua, pausa e velocidade;
5. **Ler a tela:** como identificar a camada ativa de cada dispositivo, a unidade de dados desenhada com seus cabeçalhos, o par de endereços lógicos e o par de endereços físicos vigentes;
6. **Alternar entre as pilhas:** como trocar a exibição entre as sete camadas do modelo OSI e o agrupamento TCP/IP;
7. **Provocar falhas:** como derrubar um enlace, injetar um erro de bit e enviar para um endereço inalcançável, e o que aparece no registro em cada caso;

8. **Salvar o registro de eventos:** como gravar o registro completo em arquivo e onde ele fica;
9. **Conferir um resultado conhecido:** o cenário E2 do enunciado, com os quatro quadros, os endereços físicos de cada salto e a eficiência de 11,4%, de modo que o leitor confirme que o programa funciona na máquina dele.

Observação importante: tutoriais devem ter **capturas de tela** ilustrando cada etapa. Texto puro não funciona bem como tutorial.

Exemplo de passo de tutorial

*"Na lista **Cenário**, escolha **E2 - entrega indireta** e pressione **Carregar**. O mapa passa a destacar o caminho **R1**, **R4**, **R3**, conforme a figura 3. Pressione **Passo** seis vezes: a mensagem desce as sete camadas de **H1** e o quadro **Q1** aparece sobre o enlace **H1-R1**."*

3 Documentação do Projeto

Arquivo docs/documentacao_projeto.pdf.

A documentação descreve **como o projeto funciona internamente**. É o documento que alguém leria para entender, modificar ou estender o simulador. Diferente do tutorial, aqui o foco está em **conceito** e não em execução.

Conteúdo esperado

1. **Visão geral:** o que o simulador faz e qual modelo de camadas implementa;
2. **Separação entre as camadas:** qual é a interface entre uma camada e a camada adjacente, e como se garante que nenhuma camada acesse uma camada não adjacente. Este é o eixo do projeto e merece seção própria. Explique especificamente por que o roteador não consegue ler uma porta;
3. **Diagrama de módulos:** representação visual dos componentes e do fluxo de dados, da geração da mensagem até a sua entrega ao processo de destino;
4. **Estrutura da unidade de dados:** tabela com os campos que a descrevem, seus tipos e funções. Exemplo:

Campo	Tipo	Função
dados	bytes	Conteúdo transportado
cabecalhos	lista	Cabeçalhos já acrescentados
unidade	texto	Mensagem, segmento, pacote ou quadro
logicos	tupla	Par de endereços de origem e destino
fisicos	tupla	Par de endereços do salto corrente
portas	tupla	Par de portas de origem e destino

5. **Interface dos módulos:** para cada função ou classe principal, o que recebe, o que devolve e o que faz;
6. **Parâmetros configuráveis:** tamanho de cada cabeçalho, limite de segmentação da camada 4, caminho do arquivo de topologia e velocidade de reprodução, indicando faixa válida e valor padrão de cada um;
7. **Funcionamento interno:** o que acontece a cada passo da simulação. Como a unidade de dados desce a pilha na origem, como o quadro é desmontado e outro é construído em cada salto, como a tabela de encaminhamento é consultada, como os segmentos são remontados no destino e o que acontece quando um quadro chega corrompido;
8. **Convenções de simulação:** as decisões de modelagem que determinam os números produzidos, sem as quais a mesma mensagem gera resultados diferentes. Entre elas: o tamanho de cada cabeçalho, o limite de octetos por segmento e se o cabeçalho de sessão entra em cada segmento ou uma vez só, o critério de desempate entre caminhos de mesmo custo, a numeração dos quadros e o que ocorre com o quadro descartado por erro;
9. **Dependências e ambiente:** versão do Python, bibliotecas utilizadas e como o executável foi gerado.

Tom de escrita: a documentação é escrita no **presente** e de forma **impessoal**, como uma referência técnica.

Exemplo de trecho da documentação

"Ao receber um quadro, a camada 2 confere a verificação de erro, remove o cabeçalho e o finalizador e entrega o pacote à camada 3. O quadro recebido é então descartado: o quadro de saída é um objeto novo, com outro par de endereços físicos. A camada 2 não sabe para onde o pacote vai, e a camada 3 não sabe por qual meio ele chegou."

4 Comparativo entre os documentos

A tabela abaixo resume as principais diferenças entre os três tipos de documento, para evitar confusão sobre o que colocar em cada lugar:

	README	Tutorial	Documentação
Propósito	Orientação e navegação	Reprodução passo a passo	Referência técnica
Público	Quem chega ao projeto	Quem vai usar	Quem vai modificar
Responde	O que há aqui	Como eu faço	Como funciona
Tempo verbal	Presente	Imperativo	Presente
Pessoa	Impessoal	Impessoal	Impessoal
Tamanho	Curto	Médio	Médio
Imagens	Não	Capturas de tela	Diagramas
Atualiza com o código?	Sim	Sim	Sim

Resumo rápido

- **README** = "Estou no projeto. O que tem aqui e por onde começo?"
- **Tutorial** = "Como eu executo e uso isso passo a passo?"
- **Documentação** = "Como este simulador funciona internamente?"

Dicas finais

- Comece pelo README, que é curto e organiza o resto;
- Escreva o tutorial de uso **enquanto** opera o programa, não depois. Quem escreve de memória omite justamente os passos que se tornaram automáticos;
- A documentação pode ser construída em paralelo ao desenvolvimento do código;
- Capture as telas conforme o programa fica pronto, para não precisar refazer o percurso inteiro no fim;
- Teste a documentação como quem avalia fará: descompacte a entrega em uma pasta nova, siga o próprio tutorial ao pé da letra e veja se o programa abre e produz os números esperados;
- Releia tudo no final pensando: *"alguém que nunca viu este projeto entenderia isso sem perguntar nada?"*